

CSV-Ressources for PCA

github.com/xylokopa

26. Juni 2026

1 Data-Ressources

- 1. Der Fahrzeug-Klassiker sind: „Auto-mpg“ and „mtcars“ Für das Profil (Gewicht, PS, Hubraum, Beschleunigung) ist Standard in Data Science. Variablen wie Hubraum, Leistung und Gewicht korrelieren stark miteinander (perfekt, um die Varianz auf wenige Hauptkomponenten zu reduzieren).
- Variablen : mpg (Verbrauch), cylinders, displacement (Hubraum), horsepower (PS), weight (Gewicht), acceleration (Beschleunigung), model-year (Baujahr), origin (Herkunft).
- Web Link github.com/JWarmenhoven/.../Auto.csv
- Raw Dataset 1 <https://github.com/selva86/datasets.../Auto.csv>
- Raw Dataset 2 <https://github.com/selva86/datasets.../Vehicle.csv>
- Raw Dataset 3 [raw.data 24 features](#)
- Web Link [1974er MotorTrend-Datensatz als Gist CSV \(mtcars.csv\)](#)
- Raw Dataset <https://gist.githubusercontent.com/seankross/-etc-/mtcars.csv>.
- html-Dokumentation [mtcars.csv Feature-List](#)
- 2. Der Länder-Indikatoren-Datensatz (Anonymisierte Populationsvektoren) Für das Profil von Länderstatistiken (Fläche, Population, Lebenserwartung, BIP, Inflation) gibt es einen etablierten Datensatz, der speziell für PCA- und Clustering-Aufgaben bereinigt wurde. Er zeigt eine starke Korrelation zwischen wirtschaftlichen Faktoren (BIP, Einkommen) und Gesundheitsfaktoren (Kindersterblichkeit, Lebenserwartung).
- Variablen im Datensatz: country, child-mort (Kindersterblichkeit), exports, health (Gesundheitsausgaben in percent des BIP), imports, income (Einkommen pro Kopf), inflation, life-expec (Lebenserwartung), total-fer (Geburtenrate), gdpp (BIP pro Kopf).
- World-Population 14 indicators 1962-2007: [14indicators1962-2007.csv](#)
- Stock-Market Ticker-Data: [Stock-Tickers.csv](#)
- For Python-Code: <https://www.kaggle.com/code/dgupta18/country-clusters>.
- For csv-Download: [unsupervised-learning-on-country-data](#) oder [unsupervised-learning-on-country-data?resource=download](#)

- Ergänzende geografische Daten (Breiten- und Längengrade der Länder) erhalten Sie direkt über die Google Canonical Countries [google/canonical/countries_csv](https://www.google.com/canonical/countries_csv).
- Tipp für den Schnellstart in Python (Code-Snippet) Man kann die URLs direkt mit pandas ins PCA-Skript einladen:

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
# Steuerungsvariable: True fuer Internet, False fuer lokale Datei
online = True
# Datenquelle basierend auf der Variable 'online' auswaehlen
if online:
    car_url = "https://raw.githubusercontent.com/selva86/datasets/refs/heads/master/Auto.csv"
    df_cars = pd.read_csv(car_url)
else:
    df_cars = pd.read_csv("auto.csv")
# Numerische Spalten fuer PCA auswaehlen ? korrigieren und
# standardisieren
df_cars['horsepower'] = pd.to_numeric(df_cars['horsepower'].replace('?', pd.NA), errors='coerce')
features = ['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration']
X = StandardScaler().fit_transform(df_cars[features].dropna())
# PCA berechnen
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print(f"Erklaerte Varianz der ersten beiden Komponenten: {pca.explained_variance_ratio_.sum():.2%}")
```

2 more ftp , Links and Lectures around Python and Data

1. **Python:** Stellenbosch University <https://ftp.sun.ac.za/ftp/pub/mirrors/python/>
2. **py-Index:** Python-Index <https://docs.python.org/3/py-modindex.html>
3. **gallery:** Matplotlib <https://matplotlib.org/stable/gallery/index.html>
4. **github.io:** Python DataScience [Python-Handbook](#)
5. **pub latex:** ftp.dante.de [mirror](#)
6. **pub math:** Laboratoire LIP6 <https://ftp.lip6.fr>
7. **py pdf:** SMART PARKING SYSTEMS <https://cdn.aui.ma/sse-capstone-repository>
8. **py math:** F.F.T-Implementation [towards data science](#)
9. **py math:** Fast Fourier Transform in Climate- and Geo-Physics <https://kls2177.github.io>
10. **py math:** FFT with Scipy <https://docs.scipy.org>
11. **py PCA:** World Bank Data in Human Fertility <https://www.statsmodels.org>
12. **py PCA:** What is PCA-Analysis ? <https://python-course.eu>
13. **py PCA:** Scree-Plots explained in Orange <https://orange3.readthedocs.io>
14. **py PCA:** Github-Notebook-Examples <https://github.com/erdogant/pca>
15. **py PCA:** SciKit- about PCA <https://scikit-learn.org>

16. **py PCA:** Examples in [R Markdown](#) by [Paul Jozefek](#) for PCA <https://rpubs.com/pjozefek>
17. **py PCA:** PCA around the Covid-Gap <https://fam.tuwien.ac.at>
18. **py PCA:** Covid-2020 on Github <https://github.com/lukysummer>
19. **py PCA:** Pythonization of Covid 19 on RPubS <https://rpubs.com/hansdeck>
20. **py Data:** US Covid19 Dataset <https://raw.githubusercontent.com>
21. **py Data:** Global Covid19 Dataset <https://raw.githubusercontent.com>
22. **py Data:** Covid19 Aggregated Data <https://raw.githubusercontent.com>
23. **py Data:** Application Example <https://opensource.com/article>
24. **py Data:** yahooFinance Ticker-Examples <https://finance.yahoo.com/quote/SPCX/>